

EF : CHARGEMENT DES ENTITÉS ASSOCIÉES

Cours

CHARGEMENT DES ENTITÉS ASSOCIÉES

Entity Framework prend en charge trois façons de charger les données associées:

- **Lazy Loading**
- **Eager Loading**
- **Explicit Loading**

LAZY LOADING

- Le lazy loading est le processus par lequel les données associées **sont chargées de façon transparente** à partir de la base de données **lors de l'accès à la propriété** de navigation.
- lors de l'utilisation de la classe d'entité Blog définie ci-dessous, les publications associées seront chargées lors du **premier accès à la propriété de navigation Posts**

```
public class Blog
{
    public int BlogId { get; set; }
    public string Name { get; set; }
    public string Url { get; set; }
    public string Tags { get; set; }

    public virtual ICollection<Post> Posts { get; set; }
}
```

LAZY LOADING

- Une fois les propriétés de navigation sont décorés avec le mot clé Virtual
- Installer le package Microsoft.EntityFrameworkCore.Proxies
- Activer le lazy loading en appelant la fonction UseLazyLoadingProxies() au niveau de la fonction OnConfiguring()

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder
        .UseLazyLoadingProxies()
        .UseSqlServer(@"Data Source=(localdb)\ProjectsV13;Initial Catalog=CustomerDB
}
```

DÉSACTIVER LE CHARGEMENT DIFFÉRÉ

1. Le lazy loading de la collection Posts peut être désactivé en rendant la propriété Posts **non virtuelle**:

```
public class Blog
{
    public int BlogId { get; set; }
    public string Name { get; set; }
    public string Url { get; set; }
    public string Tags { get; set; }

    public ICollection<Post> Posts { get; set; }
}
```

2. Ensuite dans le constructeur de la classe Context , on met la propriété « ChangeTracker.LazyLoadingEnabled » a False :

```
public class BloggingContextContext : DbContext
{
    0 références
    public BloggingContextContext()
    {
        this.ChangeTracker.LazyLoadingEnabled = false;
    }
}
```

EAGER LOADING

- L' Eager Loading est le processus par lequel les données associées sont chargées à partir de la base de données **dans le cadre de la requête initiale**. Le chargement hâtif est obtenu en utilisant la méthode **Include**.

```
using (var context = new BloggingContext())
{
    // Load all blogs and related posts.
    var blogs1 = context.Blogs
                        .Include(b => b.Posts)
                        .ToList();

    // Load one blog and its related posts.
    var blog1 = context.Blogs
                       .Where(b => b.Name == "ADO.NET Blog")
                       .Include(b => b.Posts)
                       .FirstOrDefault();

    // Load all blogs and related posts
    // using a string to specify the relationship.
    var blogs2 = context.Blogs
                        .Include("Posts")
                        .ToList();

    // Load one blog and its related posts
    // using a string to specify the relationship.
    var blog2 = context.Blogs
                       .Where(b => b.Name == "ADO.NET Blog")
                       .Include("Posts")
                       .FirstOrDefault();
}
```

EXPLICIT LOADING

- Avec le lazy loading désactivé, il est toujours possible de charger les entités associées, mais cela doit être fait avec un appel explicite. Pour ce faire, vous utilisez la méthode **Load** sur l'entrée de l'entité associée.
- Les entités associées sont chargées explicitement, non pas dans le cadre de la requête initiale, mais à un moment ultérieur

```
using (var context = new BloggingContext())
{
    var post = context.Posts.Find(2);

    // Load the blog related to a given post.
    context.Entry(post).Reference(p => p.Blog).Load();

    // Load the blog related to a given post using a string.
    context.Entry(post).Reference("Blog").Load();

    var blog = context.Blogs.Find(1);

    // Load the posts related to a given blog.
    context.Entry(blog).Collection(p => p.Posts).Load();

    // Load the posts related to a given blog
    // using a string to specify the relationship.
    context.Entry(blog).Collection("Posts").Load();
}
```